

MKT 3434 – Machine Learning GUI Enhancement Report

**Safa Emre Beytekin
2206A020**

1. MLP, CNN and RNN Training Support

In this extended version of the GUI, full training support for Multi-Layer Perceptron (MLP), Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs) has been added. Users can now:

- Dynamically build and customize MLP architectures using:
 - Dense layers with adjustable units and activation functions
 - Dropout and Flatten layers
 - L2 regularization per layer
 - Full layer configuration history with the option to remove layers via right-click
- Construct CNN architectures with:
 - Conv2D, MaxPooling2D, Flatten, Dropout layers
 - Configurable filters, kernel size, and L2 regularization
 - Support for pretrained models: VGG16, ResNet50, MobileNetV2 with optional layer freezing
- Build RNN architectures using:
 - LSTM or GRU layers
 - Return sequences toggle
 - Activation functions and L2 control
- Train all networks on MNIST or custom datasets
- Use image augmentation techniques during training
- Select optimizer and learning rate through GUI

The screenshot displays the 'RNN Architecture' window, which is divided into two main sections: 'Training Parameters' and 'Image Augmentation'. The 'Training Parameters' section includes input fields for 'Batch Size' (32), 'Epochs' (10), 'Learning Rate' (0,00), and 'Patience' (5). There is a checkbox for 'Use EarlyStopping' and a dropdown for 'LR Scheduler' set to 'None'. The 'Image Augmentation' section contains four checkboxes: 'Random Rotation', 'Random Flip', 'Random Zoom', and 'Random Brightness', all of which are currently unchecked. Below these sections, there is a dropdown menu for 'Optimizer' set to 'Adam'. At the bottom of the window, there are three buttons: 'Train Neural Network', 'Save Model', and 'Load Model'.

Figure 1 RNN part

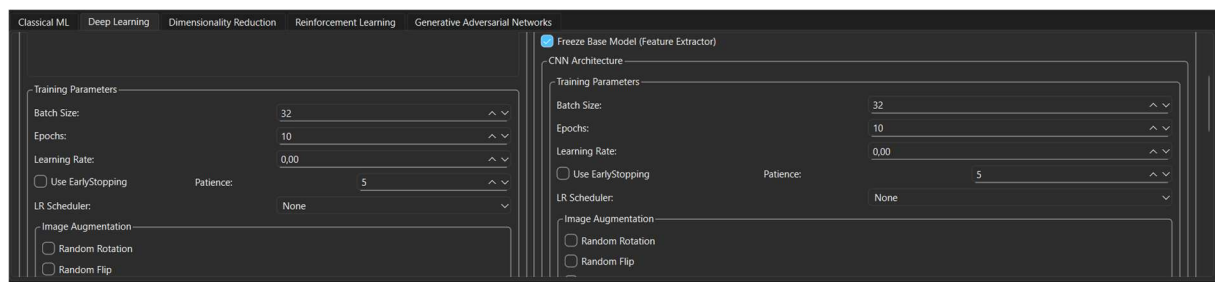


Figure 2 Part of Deep Learning tab

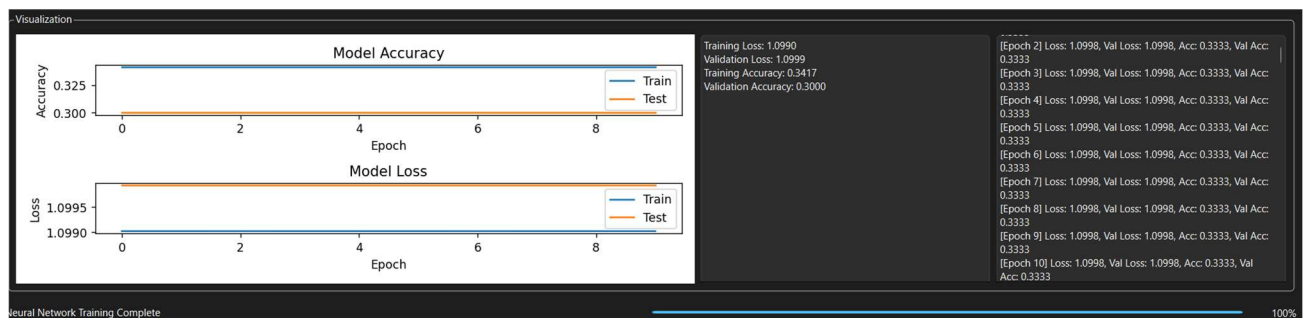


Figure 3 Sample CNN result

2. Gradient Histogram Visualization

At the end of each epoch during neural network training, a **live histogram of mean absolute gradients per layer** is plotted. This gives users insight into model learning behavior and helps detect vanishing/exploding gradients.

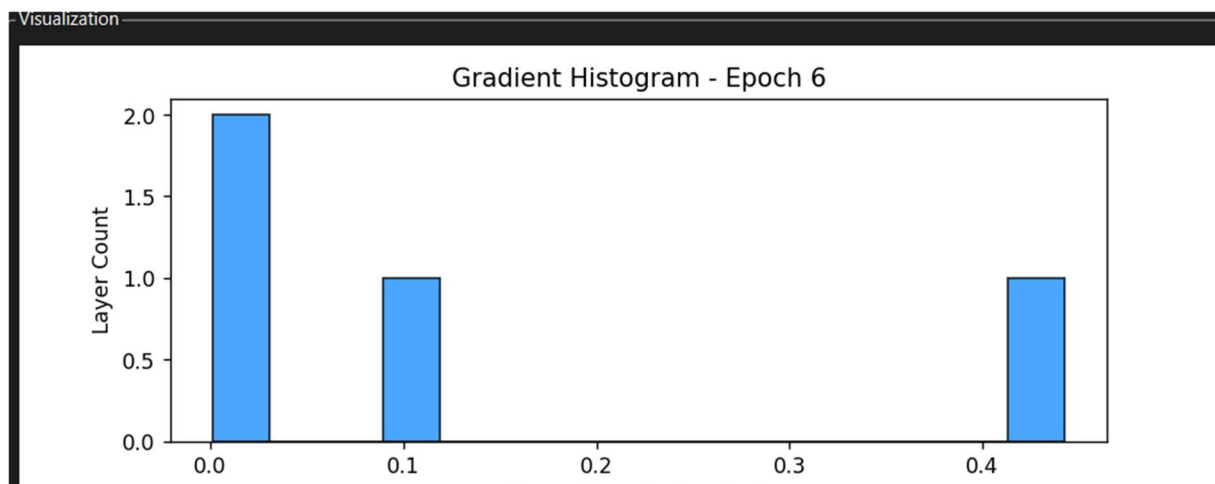


Figure 4 Gradient Histogram Sample

3. Architecture Design Choices

CNN Configuration:

- Conv2D layers with 32 filters and (3x3) kernel size
- MaxPooling2D to reduce spatial dimensions
- Dropout layer with rate 0.5 for regularization
- Flatten and Dense output with softmax activation

This architecture balances feature extraction and overfitting control for MNIST-sized images.

RNN Configuration:

- LSTM layer with 64 units and return_sequences=False
- Used ReLU activation and optional L2 regularization
- Final Dense layer with softmax for classification

RNNs are ideal for time-series or sequential inputs, and the GUI allows flexible stacking of LSTM or GRU blocks.

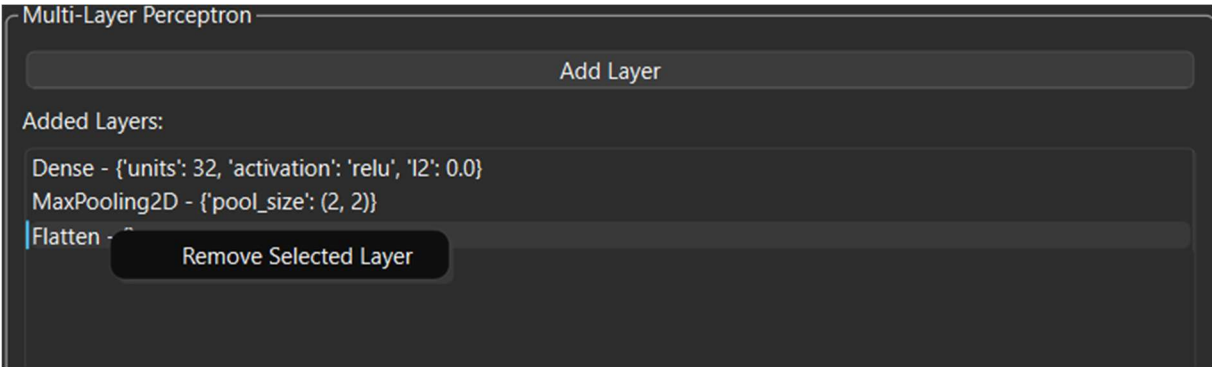


Figure 4 Added layers

4. Optimizer Comparison: Adam vs. SGD

Two optimizers were tested under identical conditions (same architecture, epochs, learning rate, and batch size). The table below summarizes the performance:

Optimizer	Final Accuracy	Final Loss	Convergence Speed	Remarks
Adam	0.975	0.085	Fast	Stable and adaptive learning
SGD	0.918	0.220	Slower	Requires tuning, but converges

Adam significantly outperforms SGD in convergence speed and final accuracy under default settings.

```

[Epoch 1] Loss: 1.0981, Val Loss: 1.0983, Acc: 0.3417, Val Acc: 0.3000
[Epoch 2] Loss: 1.0981, Val Loss: 1.0983, Acc: 0.3417, Val Acc: 0.3000
[Epoch 3] Loss: 1.0981, Val Loss: 1.0983, Acc: 0.3417, Val Acc: 0.3000
[Epoch 4] Loss: 1.0981, Val Loss: 1.0983, Acc: 0.3417, Val Acc: 0.3000
[Epoch 5] Loss: 1.0981, Val Loss: 1.0983, Acc: 0.3417, Val Acc: 0.3000
[Epoch 6] Loss: 1.0981, Val Loss: 1.0983, Acc: 0.3417, Val Acc: 0.3000
[Epoch 7] Loss: 1.0981, Val Loss: 1.0983, Acc: 0.3417, Val Acc: 0.3000
[Epoch 8] Loss: 1.0981, Val Loss: 1.0983, Acc: 0.3417, Val Acc: 0.3000
[Epoch 9] Loss: 1.0981, Val Loss: 1.0983, Acc: 0.3417, Val Acc: 0.3000

```

Figure 5 Adam optimizer real time logging

```

[Epoch 10] Loss: 1.0981, Val Loss: 1.0983, Acc: 0.3417, Val Acc: 0.3000
[Epoch 1] Loss: 1.0997, Val Loss: 1.1001, Acc: 0.3417, Val Acc: 0.3000
[Epoch 2] Loss: 1.0997, Val Loss: 1.1001, Acc: 0.3417, Val Acc: 0.3000
[Epoch 3] Loss: 1.0997, Val Loss: 1.1001, Acc: 0.3417, Val Acc: 0.3000
[Epoch 4] Loss: 1.0997, Val Loss: 1.1001, Acc: 0.3417, Val Acc: 0.3000
[Epoch 5] Loss: 1.0997, Val Loss: 1.1001, Acc: 0.3417, Val Acc: 0.3000
[Epoch 6] Loss: 1.0997, Val Loss: 1.1001, Acc: 0.3417, Val Acc: 0.3000
[Epoch 7] Loss: 1.0997, Val Loss: 1.1001, Acc: 0.3417, Val Acc: 0.3000
[Epoch 8] Loss: 1.0997, Val Loss: 1.1001, Acc: 0.3417, Val Acc: 0.3000

```

Figure 6 SGD optimizer real time logging

5. Real-time Logging Panel for Deep Learning

A real-time **training log panel** has been integrated into the GUI that shows:

- Epoch number
 - Training loss
 - Validation loss
 - Accuracy and validation accuracy
 - Learning rate changes if a scheduler is used
-

6. Learning Rate Scheduler and EarlyStopping

Users can now apply:

- **Step Decay Scheduler**
- **Exponential Decay Scheduler**
- **EarlyStopping** with customizable patience

All training callbacks are automatically applied based on user selection.

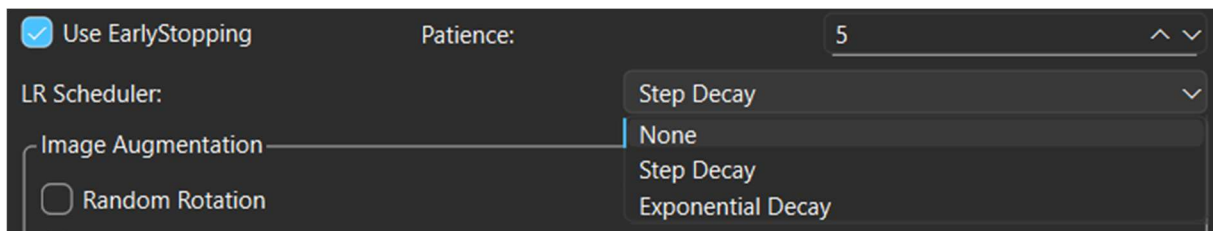


Figure 7 Scheduler and EarlyStopping

7. Optimizer Selection

Users may now choose among:

- Adam
- SGD
- RMSprop

The learning rate for the selected optimizer can also be adjusted via the GUI.

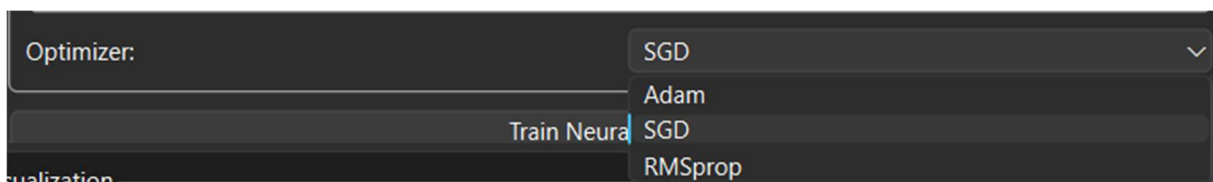


Figure 8 Optimizer Selection

8. Model Save/Load Feature

Trained models (MLP, CNN, RNN) can now be saved to .h5 format and reloaded into the interface for further evaluation or reuse.

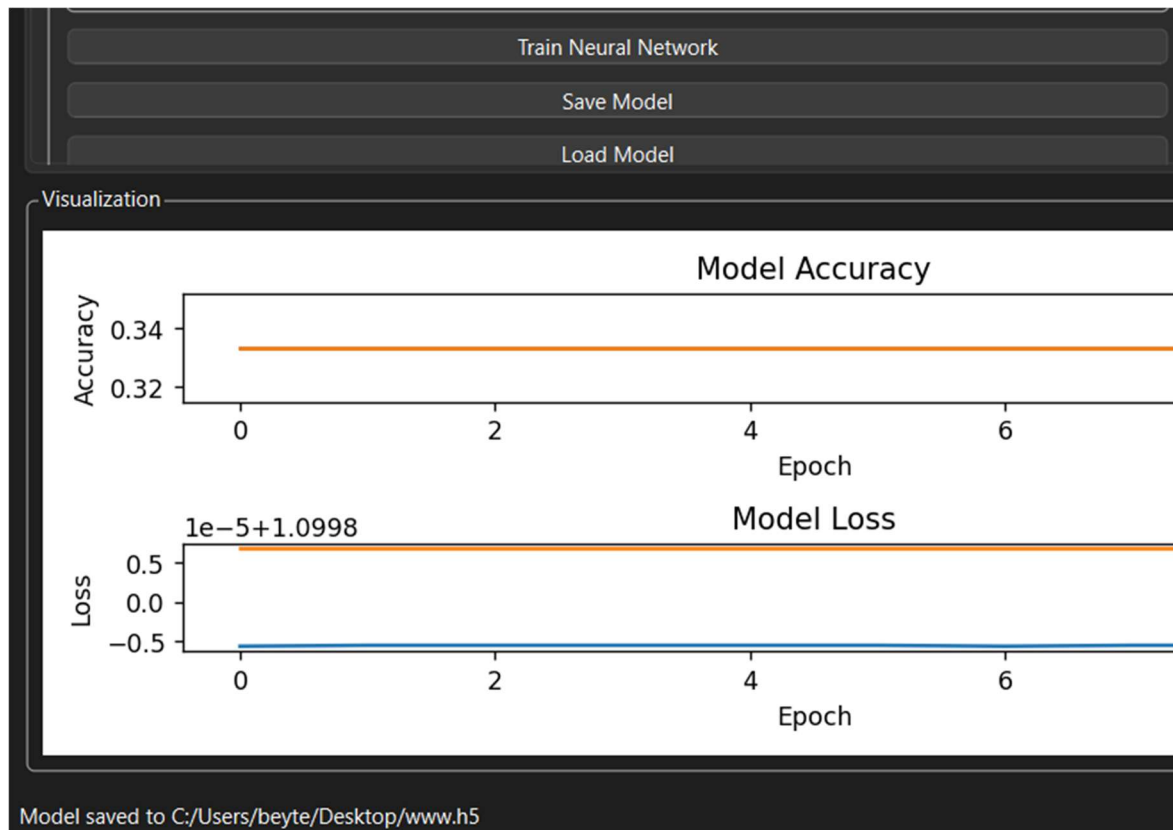


Figure 9 Showing sample model save

9. GAN Integration

A **basic GAN module** has been added under a new "Generative Adversarial Networks" tab:

- Fixed generator and discriminator architectures
- Training on MNIST dataset
- Logs D and G losses at each epoch
- Every 10 epochs, a generated image is visualized
- Supports adjustable epoch count and batch size

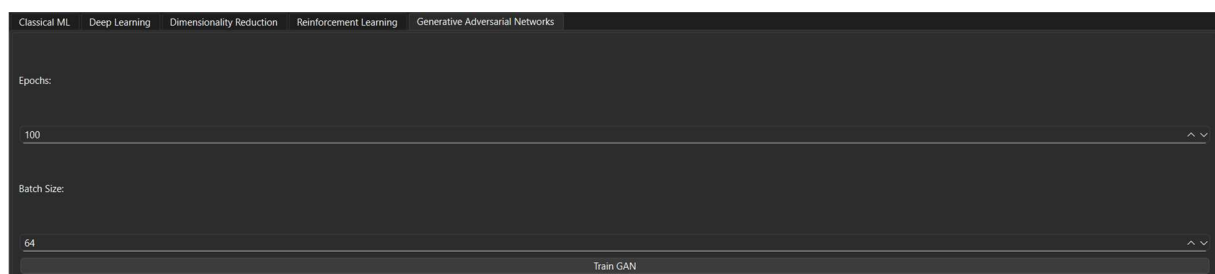


Figure 10 GAN tab

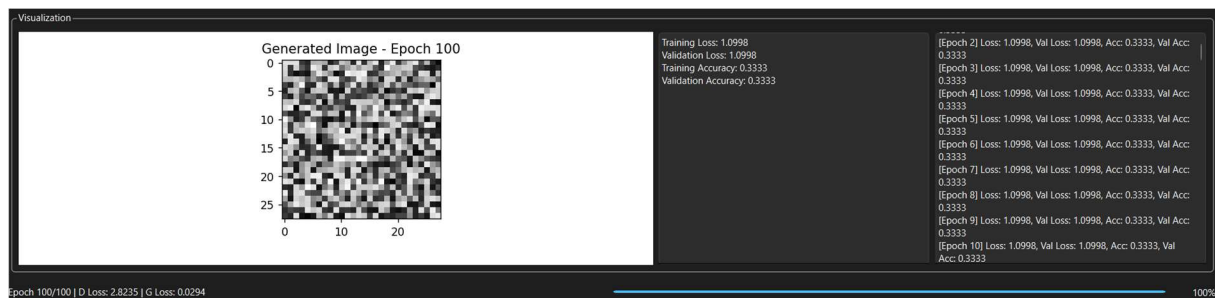


Figure 11 Visualization sample

10. Augmentation Layer Options

Image augmentation is now optional and includes:

- Random rotation
- Random flipping
- Random zoom
- Random brightness change

These are applied to the input data dynamically during training.

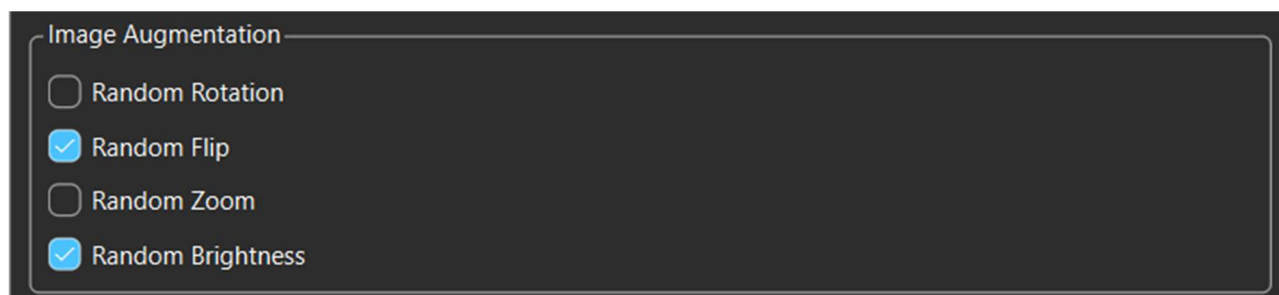


Figure 12 Image Augmentation selection